

Week 01: Artificial Intelligence in Engineering: Agents and First Steps in Python

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-01/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-01/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-01/lab.html>

Colab notebook with Python step 1 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-01/NB01_first_agent.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Engineering has always delegated decisions to machines: A governor regulates the speed of a steam engine, a thermostat switches a boiler and an autopilot holds an altitude. Artificial intelligence extends this delegation to decisions that need perception, reasoning, search, learning or language [1, 2]. The week defines artificial intelligence through the idea of a rational agent, introduces the vocabulary used for the rest of the semester, and places the families of techniques covered in the course on one map. The practical work builds a first agent in Python: a heating controller that keeps a room in Isparta comfortable during a real January, driven by reanalysis weather data [3, 4].

Estimated study time: 8 to 10 hours.

Learning outcomes

By the end of the week, students are expected to define artificial intelligence from the rational-agent perspective, to outline its history, to place machine learning, deep learning and intelligent optimisation within its family and relate it to neighbouring fields, to describe an engineering system with the PEAS scheme and classify its environment, to distinguish reflex, model-based, goal-based, utility-based and learning agents, to explain why hysteresis turns a simple reflex into an agent with memory, to name the main environments for Python programming, and to run, modify and evaluate a short Python simulation in Google Colab.

Python in this week

These notes close with Python step 1: Values, variables and decisions. It covers values, variables and types, arithmetic, f-strings, comparisons and if statements, applied to the heating rule of the week's agent [5]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

What engineers mean by artificial intelligence

In 1950, Alan Turing replaced the question of whether machines can think with a behavioural test: Can a machine answer questions in a way that an interrogator cannot tell apart from a person [6]? Five years later, the proposal for the Dartmouth summer project used the phrase artificial intelligence and stated the working conjecture of the new field: Every aspect of learning or any other feature of intelligence can in principle be described so precisely that a machine can be made to simulate it [7]. Both documents still shape the field, but neither gives an engineer a design criterion.

Russell and Norvig organise definitions of artificial intelligence along two axes: Whether the aim is to think or to act, and whether the standard is human performance or rationality [1]. Engineering practice mostly adopts the fourth combination, acting rationally. A rational agent selects, for every sequence of perceptions, the action that is expected to maximise a performance measure, given what it knows. This definition is useful because it can be tested. It asks what the system perceives, which actions it can take, and how success is measured, and these are questions that every engineering specification already answers.

Machine learning is one route to rational behaviour, not a synonym for artificial intelligence. Jordan and Mitchell describe machine learning as the study of systems that improve their performance on a task through experience [2]. A route planner that uses a hand-written heuristic, a fuzzy controller written from operator knowledge and a genetic algorithm that designs a truss are all artificial intelligence techniques without being learning systems in this sense. The course treats the whole toolbox, because engineering problems differ in how much data, knowledge and computing time they offer.

Check your understanding. Which definition does engineering practice mostly adopt?

- A. Thinking humanly: The system must reproduce human thought processes
- B. Acting humanly: The system must pass a Turing-style test
- C. Acting rationally: The system selects actions expected to maximise a performance measure
- D. Any system that contains a neural network

Answer: C. The rational-agent view turns intelligence into a testable design criterion: percepts, actions and a performance measure.

Formulas: The rational agent

At every time step t the agent receives a percept p_t , such as a temperature reading. The agent function f maps the whole sequence of percepts received so far to an action a_t [1]:

$$a_t = f(p_1, p_2, \dots, p_t)$$

The dots stand for all percepts between the first and the latest one. An action rarely has a certain outcome. Each possible outcome s of action a has a probability $P(s|a)$, read as “the probability of s given a ”, and a utility $U(s)$, a number that states how good the outcome is according to the performance measure. Because some outcome must occur, the probabilities of the outcomes of one action add up to one:

$$\sum_s P(s|a) = 1$$

The Greek capital letter sigma, Σ , with s written below it means “add the following term for every possible outcome s ”. The expected utility of an action is the average of the utilities of its outcomes, each weighted by its probability:

$$E(U|a) = \sum_s P(s|a) U(s)$$

A rational agent chooses the action with the largest expected utility [1]:

$$a^* = \arg \max_a E(U|a)$$

The operator $\arg \max$ returns the argument at which an expression is largest, here the action itself, not the largest value. The star marks the best choice.

Worked example: A heating decision under uncertainty

At 06:00 a heating agent decides whether to run the heater during the next hour. The outcome depends on the weather and on how the occupants behave, so the agent works with probabilities. Comfort is scored from 0 to 10, and the energy cost of heating is expressed on the same scale, so that the utility of an outcome is its comfort score minus the energy cost.

Action a	Outcome s	$P(s a)$	Comfort score	Energy cost	Utility $U(s)$
heat	comfortable	0.9	10	3	7
heat	too warm	0.1	6	3	3
off	comfortable	0.3	10	0	10
off	too cold	0.7	2	0	2

Step 1. Utilities. Each utility is the comfort score minus the energy cost:

$$U(\text{heat, comfortable}) = 10 - 3 = 7, U(\text{heat, too warm}) = 6 - 3 = 3$$

$$U(\text{off, comfortable}) = 10 - 0 = 10, U(\text{off, too cold}) = 2 - 0 = 2$$

Step 2. Probabilities. Each action has two outcomes, and their probabilities add up to one: $0.9 + 0.1 = 1$ and $0.3 + 0.7 = 1$.

Step 3. Expected utility of heating.

$$E(U|\text{heat}) = 0.9 \times 7 + 0.1 \times 3 = 6.3 + 0.3 = 6.6$$

Step 4. Expected utility of leaving the heater off.

$$E(U|\text{off}) = 0.3 \times 10 + 0.7 \times 2 = 3 + 1.4 = 4.4$$

Step 5. Decision. Since $6.6 > 4.4$, the rational choice is $a^* = \text{heat}$.

Step 6. Break-even energy cost. Let the energy cost be an unknown c . The expected comfort of heating is $0.9 \times 10 + 0.1 \times 6 = 9.6$, and the cost is paid in both outcomes, so

$$E(U|\text{heat}) = 0.9(10 - c) + 0.1(6 - c) = 9.6 - c$$

Heating stays the better choice as long as $9.6 - c > 4.4$, that is, as long as $c < 9.6 - 4.4 = 5.2$. If the energy cost rises to 6 units, then $E(U|\text{heat}) = 9.6 - 6 = 3.6$, which is smaller than 4.4, and the rational choice becomes “off”.

The same agent therefore changes its decision when energy prices change, without any change to its program. A utility-based agent makes such trade-offs explicit and testable. Part A of the [interactive lab](#) repeats this calculation with any probabilities, comfort scores and energy costs.

A short history of artificial intelligence

The field took shape between 1943 and 1956. McCulloch and Pitts described the neuron as a logical switching element [8], Turing proposed his behavioural test [6], and the Dartmouth proposal of 1956 gave the field its name and a research programme for a summer workshop held in 1956 [7]. Samuel's checkers program, which improved by playing games against itself, introduced the term machine learning [9], and Rosenblatt's perceptron learned to classify patterns from examples [10].

Symbolic methods dominated the following two decades: Search guided by heuristics, such as the A* algorithm [11], logical reasoning and, from the 1970s, expert systems that stored the knowledge of specialists as rules [12]. Zadeh introduced fuzzy sets in 1965 [13], and Holland developed genetic algorithms [14]. Minsky and Papert showed what single-layer perceptrons cannot compute [15]. Expectations repeatedly ran ahead of results, and funding and interest fell sharply in the 1970s and again in the late 1980s, after the commercial boom of expert systems. These periods are known as AI winters [1].

From the mid-1980s, learning from data moved to the centre. Backpropagation made multilayer networks trainable [16], support vector machines offered learning with strong theoretical guarantees [17], and statistical learning matured into a coherent discipline [18]. Nature-inspired optimisation grew with particle swarms [19] and ant colonies [20]. In 1997 the chess computer Deep Blue defeated the world champion Garry Kasparov [21], and the long short-term memory network appeared, later a standard model for sequences [22].

Since 2012, deep learning has dominated the field. A deep convolutional network won the ImageNet recognition challenge of 2012 by a wide margin [23], made possible by large labelled datasets, graphics processors and better training methods [24]. AlphaGo, published in 2016, combined deep networks, tree search and reinforcement learning and became the first program to defeat a professional player at the full game of Go [25]. The transformer architecture of 2017 [26] led to large language models [27] and to language models trained to follow instructions with human feedback, the approach behind the chat assistants that spread from late 2022 [28]. Scientific and engineering applications followed, among them protein structure prediction [29] and medium-range weather forecasting [30]. The timeline below places these milestones side by side.

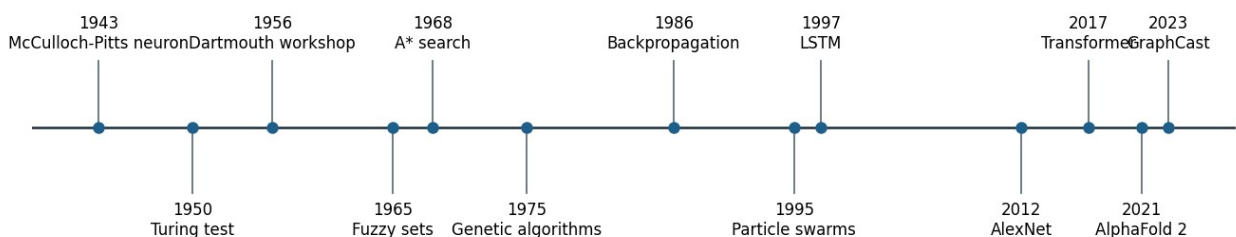


Figure 1.1. Selected milestones of artificial intelligence that recur in this course [6, 7, 8, 11, 13, 14, 16, 19, 22, 23, 26, 29, 30].

The history carries a lesson for engineers. Neural networks, search, fuzzy logic and evolutionary methods are all decades old; what changed over time was the availability of data, computing power and reliable software. A method that fails today for lack of data may succeed later, and a fashionable method may be the wrong tool for a problem with little data and much physical knowledge.

Check your understanding. Neural networks were proposed in the 1940s and 1950s. Why did deep networks come to dominate only after 2012?

- A. Their mathematical basis was discovered only in 2012
- B. Large labelled datasets, graphics processors and better training methods made deep networks

practical

C. Symbolic methods were abandoned after the AI winters

D. Fuzzy logic removed the need for data

Answer: B. The core ideas were old. What changed was the availability of data, computing power and training techniques.

The family of artificial intelligence

Artificial intelligence is an umbrella term for a family of methods, and the relations between its members are easiest to see as sets. Machine learning is a subset of artificial intelligence: It covers systems that improve their performance on a task through experience [2]. Deep learning is a subset of machine learning that uses neural networks with many layers to learn representations directly from raw data such as images, signals and text [24, 31]. Generative models and large language models form a further subset of deep learning [26, 27]. Reinforcement learning, which learns to act from rewards, belongs to machine learning and overlaps with deep learning wherever deep networks represent the policy or the value function [25, 32].

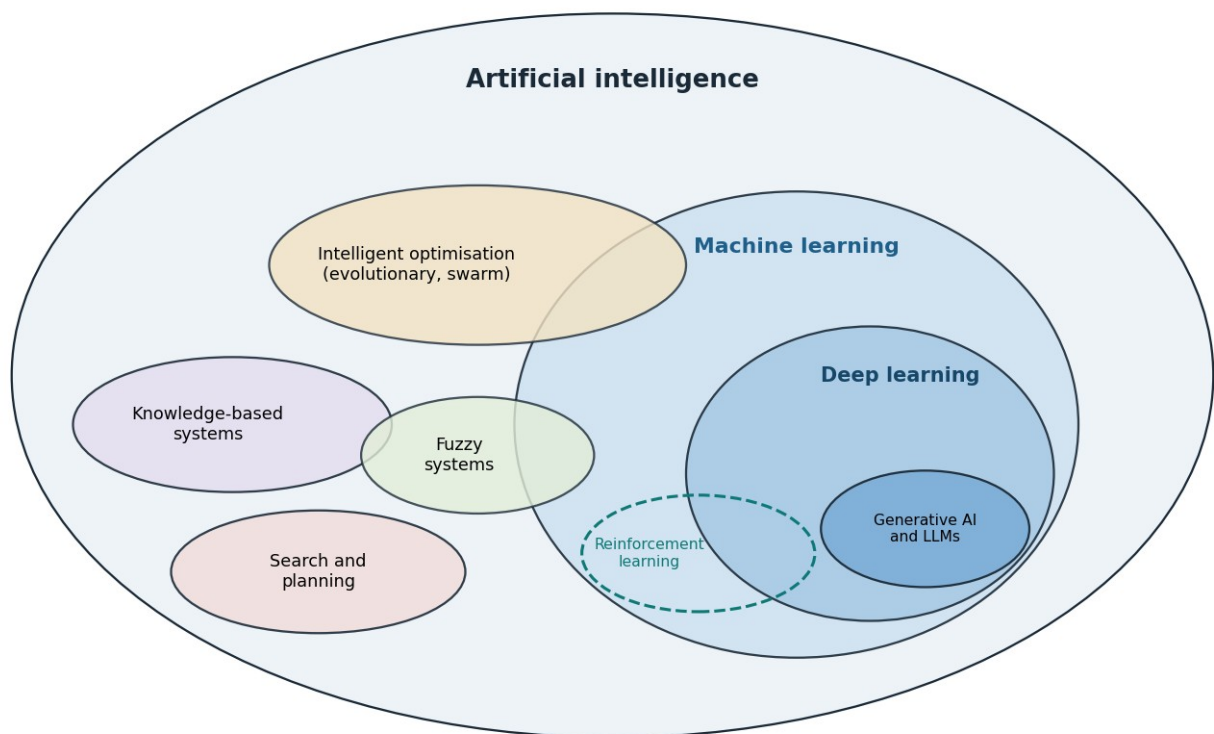


Figure 1.2. The family of artificial intelligence drawn as sets. Every deep learning method is a machine learning method, and every machine learning method is an artificial intelligence method, but the reverse does not hold [1, 24, 33].

Several members of the family do not learn from data in this sense. Search and planning compute sequences of actions from a model of the problem [11], knowledge-based systems reason with rules written by experts [12], and fuzzy systems turn vague linguistic knowledge into numbers [13]. Intelligent optimisation, also called nature-inspired optimisation, searches large design spaces with populations of candidate solutions that are varied and selected. Evolutionary algorithms and swarm intelligence are its two main branches [14, 19, 20]. Together with neural networks and fuzzy systems, these methods are often grouped as computational intelligence [33]. The sets overlap where methods combine: Evolutionary algorithms tune the settings of learning models or evolve neural networks, neuro-fuzzy systems learn fuzzy rules from data, and fuzzy rule bases are themselves a kind of knowledge-based system.

The figure also corrects two common misreadings. Artificial intelligence is neither the same as deep learning nor the same as chat assistants, which occupy a small corner of the family. The weeks of this course move through the whole family, as the map of the course below shows.

Check your understanding. Which statement about the family of artificial intelligence is correct?

- A. Every artificial intelligence method learns from data
- B. Every deep learning method is also a machine learning method
- C. Particle swarm optimisation is a kind of deep learning
- D. Machine learning and artificial intelligence are two names for the same field

Answer: B. Deep learning is a subset of machine learning, which is a subset of artificial intelligence. Many methods of the family, such as search, rules and swarms, do not learn from data.

Artificial intelligence and neighbouring fields

Artificial intelligence is a field of computer science, but its methods grew from, and feed back into, many other disciplines [1]. Statistics shares the problem of learning from data, and the two meet in statistical learning [18]. Mathematical optimisation and operations research formulate the design and scheduling problems that search and nature-inspired methods solve. Control engineering shares the question of how to act on a dynamic system, and reinforcement learning and fuzzy control lie in the overlap [32]. Neuroscience and psychology inspired neural networks from the first neuron model onwards [8, 10]. Linguistics meets artificial intelligence in natural language processing, signal and image processing meets it in speech recognition and computer vision, robotics in perception and motion planning, and data science in the handling of large and messy datasets.

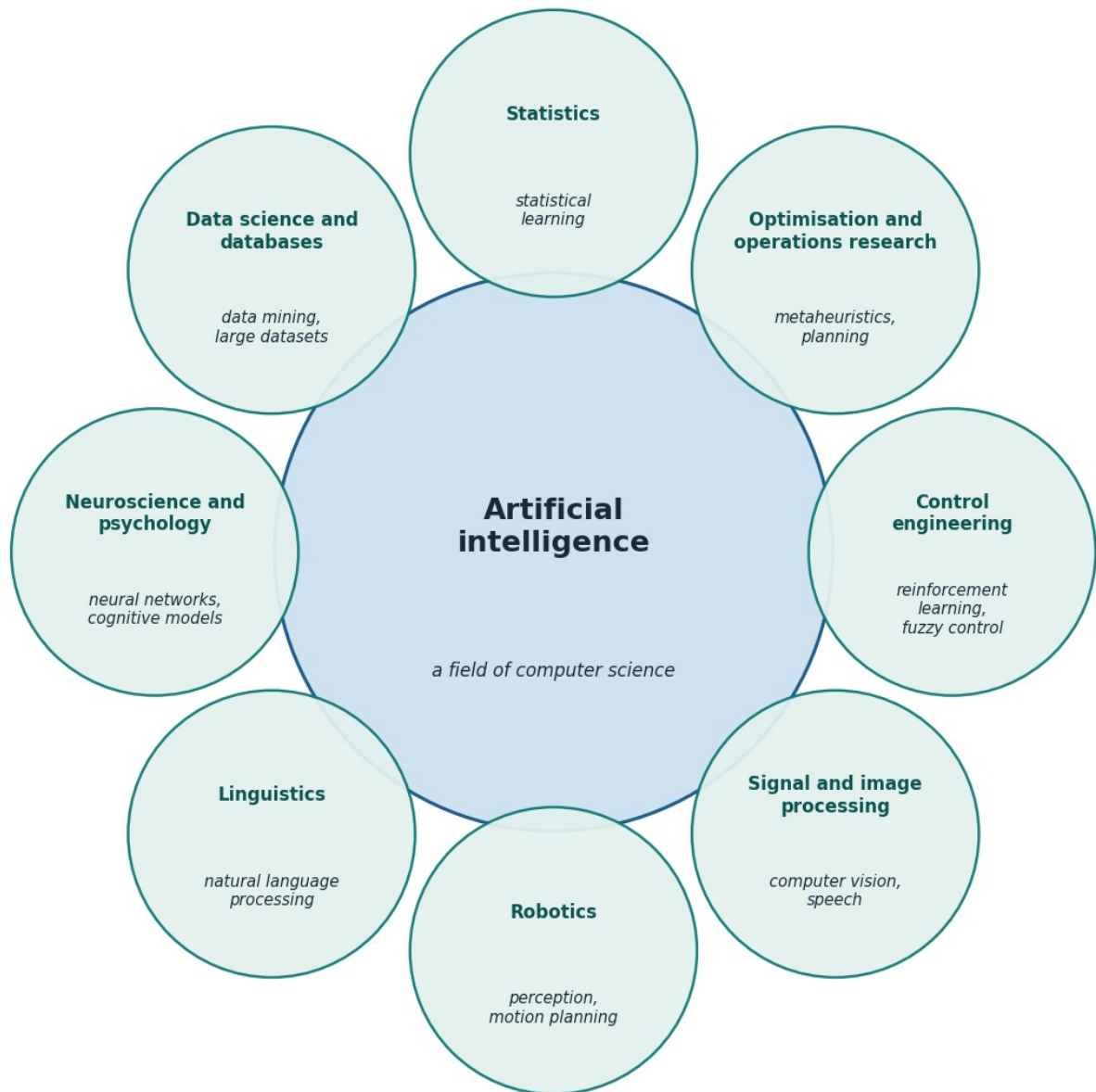


Figure 1.3. Artificial intelligence and its neighbouring fields. Each circle names a field and, below the name, the topics that it shares with artificial intelligence [1].

For an engineer these overlaps are practical. A problem often has a well-tested solution in a neighbouring field, such as a classical controller, a regression model with confidence intervals or a linear program. A new artificial intelligence method is justified only when it does better on the performance measure that matters.

Check your understanding. A team predicts the compressive strength of concrete from the mix proportions and reports confidence intervals for its predictions. In which overlap does this work lie?

- A. Artificial intelligence and linguistics
- B. Artificial intelligence and statistics
- C. Artificial intelligence and robotics
- D. Artificial intelligence and neuroscience

Answer: B. Learning a predictive model from data and quantifying its uncertainty is the shared ground of machine learning and statistics.

Agents, environments and performance measures

An agent perceives its environment through sensors and acts on it through actuators [1, 34]. The mapping from percept histories to actions is the agent function, and the program that implements it runs on some physical architecture. The PEAS scheme makes the design task explicit by listing the performance measure, the environment, the actuators and the sensors. The table below applies the scheme to four systems from different departments.

System	Performance measure	Environment	Actuators	Sensors
Heating controller of a building	Comfort band held, energy used, number of switching cycles	Rooms, walls, outdoor weather, occupants	Boiler relay, valves	Room and outdoor thermometers
Autonomous haul truck in an open-pit mine	Tonnes moved per hour, zero collisions, fuel use	Haul roads, loaders, other vehicles, dust	Steering, throttle, brakes, dump bed	GNSS, lidar, radar, payload scale
Fabric inspection camera in a weaving mill	Defects found, false alarms, line speed kept	Moving fabric, lighting, loom vibration	Alarm, marking device, line stop	Line-scan camera, encoder
Earthquake early-warning agent	Warning time, missed events, false alerts	Seismic network, communication links	Alerts to trains, factories, phones	Seismometers, accelerometers

The environment decides which techniques are feasible. It is fully observable when the sensors reveal the complete relevant state, and partially observable otherwise. It is deterministic when the next state follows from the current state and action, and stochastic when chance intervenes. It is episodic when each decision stands alone, as in inspecting one fabric image, and sequential when decisions affect the future, as in heating a building. It is static or dynamic, discrete or continuous, and it contains one agent or several [1]. A heating controller works in a partially observable, stochastic, sequential, dynamic and continuous environment: It cannot measure the heat stored in the walls, the weather is uncertain, and every switching decision changes the temperatures it will face later.

Check your understanding. A camera classifies each fabric image as defective or not, and one decision does not influence the next image. How is this task environment best described?

- A. Episodic
- B. Sequential
- C. Multi-agent
- D. Fully deterministic and continuous by definition

Answer: A. Each inspection is a separate episode. Heating a building is sequential, because each action changes future states.

Formulas: Performance measures of a heating controller

A performance measure turns the goals of the designer into numbers. For a heating controller observed over N time steps, let T_k be the room temperature at step k , T_{set} the set point and u_k the heater state, with $u_k = 1$ when the heater is on and $u_k = 0$ when it is off. The mean absolute error measures comfort; the vertical bars denote the absolute value, which turns negative deviations into positive ones:

$$\text{MAE} = \frac{1}{N} \sum_{k=1}^N |T_k - T_{\text{set}}|$$

The energy used by a heater of power P in kilowatts during steps of length Δt in hours is

$$E = \sum_{k=1}^N P u_k \Delta t$$

in kilowatt hours, because only the steps with $u_k = 1$ contribute. Each change of the heater state wears the relay and the boiler. With u_0 the state before the observed period, the number of switches is

$$S = \sum_{k=1}^N |u_k - u_{k-1}|$$

since $|u_k - u_{k-1}|$ equals 1 exactly when the state changes. A single score combines the three goals with weights λ_E and λ_S (lambda) that express how much one kilowatt hour and one switch matter compared with one degree of error [1]:

$$J = \text{MAE} + \lambda_E E + \lambda_S S$$

A lower J is better. Choosing the weights is a design decision, not a property of the algorithm.

Worked example: Scoring half an hour of control

A controller with set point $T_{\text{set}} = 21^\circ \text{C}$ and a 3 kW heater is observed for six steps of 5 minutes. The heater was off before the period, so $u_0 = 0$. The table lists the readings and the two quantities that each row contributes.

k	T_k ($^\circ\text{C}$)	u_k	$ T_k - T_{\text{set}} $	$ u_k - u_{k-1} $
1	20.3	1	$ 20.3 - 21 = 0.7$	$ 1 - 0 = 1$
2	20.6	1	$ 20.6 - 21 = 0.4$	$ 1 - 1 = 0$
3	21	1	$ 21 - 21 = 0$	$ 1 - 1 = 0$
4	21.4	1	$ 21.4 - 21 = 0.4$	$ 1 - 1 = 0$
5	21.7	0	$ 21.7 - 21 = 0.7$	$ 0 - 1 = 1$
6	21.3	0	$ 21.3 - 21 = 0.3$	$ 0 - 0 = 0$

Step 1. Mean absolute error. The sum of the deviations is

$$0.7 + 0.4 + 0 + 0.4 + 0.7 + 0.3 = 2.5$$

and dividing by $N = 6$ gives $\text{MAE} = 2.5/6 = 0.4167^\circ \text{C}$.

Step 2. Energy. A step of 5 minutes lasts $\Delta t = 5/60 = 1/12$ hour. The heater was on in $1 + 1 + 1 + 1 + 0 + 0 = 4$ steps, so

$$E = 3 \times \frac{1}{12} \times 4 = \frac{12}{12} = 1 \text{ kWh}$$

Step 3. Switches. The last column of the table adds up to $S = 1 + 0 + 0 + 0 + 1 + 0 = 2$: The heater switched on at the first step and off at the fifth.

Step 4. Combined score. With $\lambda_E = 0.2$ per kWh and $\lambda_S = 0.05$ per switch,

$$J = 0.4167 + 0.2 \times 1 + 0.05 \times 2 = 0.4167 + 0.2 + 0.1 = 0.7167$$

Two controllers can now be compared with one number, and the weights show openly which compromise the designer has chosen. The scoreboard of Part B of the [interactive lab](#) reports the same three measures for a whole week, and Part C asks for the environment properties of six engineering systems and explains each answer.

Agent architectures: From reflexes to learning

Russell and Norvig describe five agent designs of increasing capability [1]. A simple reflex agent maps the current percept to an action through condition-action rules, such as switching the heater on when the room is below the set point. A model-based reflex agent keeps an internal state that summarises what it cannot currently see. A goal-based agent considers future consequences of actions and chooses those that achieve a goal, which is the topic of search in Week 2. A utility-based agent compares outcomes with a numerical utility, for example trading comfort against energy cost. A learning agent improves any of these components from experience, which is the subject of Weeks 7 to 14.

Control engineering developed many of the same ideas under different names. Wiener's cybernetics treated the animal and the machine as feedback systems [35], and feedback control remains the most widespread form of automated decision making in engineering [36]. The thermostat illustrates the connection. A pure reflex rule switches the heater whenever the temperature crosses the set point, so noise and delays make the relay chatter on and off many times per hour. Adding a dead band, in which the heater turns on below the lower threshold and off above the upper threshold and otherwise keeps its previous state, gives the controller one bit of memory. The agent is no longer a function of the current percept alone: It is a model-based reflex agent, and the price for fewer switching cycles is a wider temperature swing.

Animation: Hysteresis in a heating agent. The animation simulates one winter day. Narrow the dead band to zero and watch the number of switching cycles rise, then widen it and watch the temperature swing grow. Run it on the [lecture page](#).

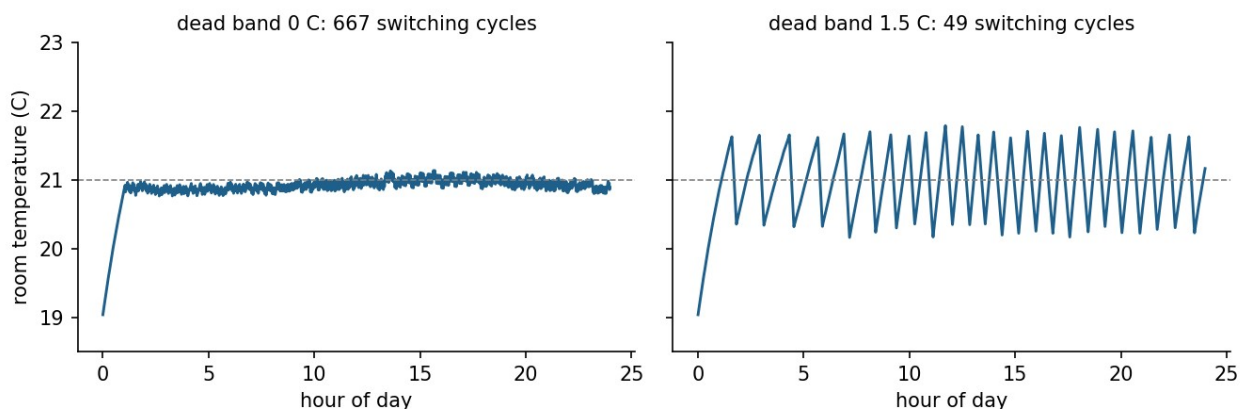


Figure 1.4. Room temperature under a simple reflex rule and a hysteresis rule on the same synthetic winter day. The dead band trades switching cycles against comfort.

Check your understanding. Why is a thermostat with a dead band a model-based reflex agent rather than a simple reflex agent?

- A. It learns the thermal model of the room from data
- B. Its action depends on an internal state, the previous heater state, as well as on the current temperature
- C. It plans a sequence of future actions with search
- D. It uses a neural network to predict the weather

Answer: B. Inside the dead band the action is whatever it was before, so one bit of memory enters the decision.

Formulas: The room model and the thermostat rules

The room of this week is a first-order thermal system. Heat leaks to the outside at a rate proportional to the temperature difference, and the heater adds heat at a constant rate while it is on [36]:

$$\frac{dT}{dt} = \frac{T_{\text{out}} - T}{\tau} + r u$$

Here dT/dt is the rate of change of the room temperature in °C per minute, T_{out} is the outdoor temperature, τ (tau) is the time constant of the room in minutes, which grows with better insulation, r is the warming rate of the heater in °C per minute, and u is the heater command, 1 for on and 0 for off. A computer advances the model in small steps of length Δt by replacing the derivative with the change over one step, the explicit Euler method:

$$T_{k+1} = T_k + \Delta t \left(\frac{T_{\text{out}} - T_k}{\tau} + r u_k \right)$$

If the heater state and the outdoor temperature stay constant, the room settles at the temperature where the rate of change is zero:

$$0 = \frac{T_{\text{out}} - T_{\infty}}{\tau} + r u \Rightarrow T_{\infty} = T_{\text{out}} + r \tau u$$

The arrow reads “which gives”. The model then has an exact solution, in which T_0 is the temperature at time $t=0$ and e is Euler’s number, about 2.71828:

$$T(t) = T_{\infty} + (T_0 - T_{\infty}) e^{-t/\tau}$$

The difference to the final temperature shrinks by the factor e^{-1} , about 0.37, every τ minutes. Solving the exact solution for t with the natural logarithm $\ln$ gives the time needed to reach a target temperature:

$$t = \tau \ln \left(\frac{T_0 - T_{\infty}}{T_{\text{target}} - T_{\infty}} \right)$$

The simple reflex agent switches on below the set point:

$$u_k = 1 \text{ if } T_k < T_{\text{set}}, \text{ otherwise } u_k = 0$$

The hysteresis agent uses a dead band of width b with a lower and an upper threshold:

$$T_L = T_{\text{set}} - \frac{b}{2}, T_H = T_{\text{set}} + \frac{b}{2}$$

$$u_k = 1 \text{ if } T_k < T_L, u_k = 0 \text{ if } T_k > T_H, u_k = u_{k-1} \text{ otherwise}$$

The last case is the one bit of memory that makes it a model-based reflex agent. With constant outdoor temperature, the room heats from T_L to T_H towards $T_{\infty, \text{on}} = T_{\text{out}} + r \tau$ and cools from T_H to T_L towards $T_{\infty, \text{off}} = T_{\text{out}}$. The time formula gives the two phases of one cycle:

$$t_{\text{on}} = \tau \ln \left(\frac{T_L - T_{\infty, \text{on}}}{T_H - T_{\infty, \text{on}}} \right), t_{\text{off}} = \tau \ln \left(\frac{T_H - T_{\infty, \text{off}}}{T_L - T_{\infty, \text{off}}} \right)$$

A cycle lasts $t_c = t_{\text{on}} + t_{\text{off}}$ minutes. A day of 1440 minutes then contains $1440/t_c$ cycles and twice as many switches, the heater is on for the fraction $D = t_{\text{on}}/t_c$ of the time, called the duty cycle, and a heater of power P uses $E_{\text{day}} = 24 D P$ kilowatt hours per day.

Worked example: Three steps of the room model

The room starts at $T_0 = 18.4^\circ \text{C}$ with $T_{\text{out}} = -3.5^\circ \text{C}$, $\tau = 240$ minutes, $r = 0.12^\circ \text{C}$ per minute, $\Delta t = 5$ minutes and the heater on, $u_k = 1$. These are the values of Python step 1. Intermediate results are rounded to six decimal places, and every step continues with the rounded value, so that the calculation can be repeated exactly with a calculator.

Step 1, from T_0 to T_1 . The four parts of the formula are computed one after the other:

$$T_{\text{out}} - T_0 = -3.5 - 18.4 = -21.9$$

$$\frac{-21.9}{240} = -0.09125, -0.09125 + 0.12 \times 1 = 0.02875$$

$$T_1 = 18.4 + 5 \times 0.02875 = 18.4 + 0.14375 = 18.54375$$

Step 2, from T_1 to T_2 . The same four operations are repeated with the new temperature:

$$T_{\text{out}} - T_1 = -3.5 - 18.54375 = -22.04375$$

$$\frac{-22.04375}{240} = -0.091849, -0.091849 + 0.12 \times 1 = 0.028151$$

$$T_2 = 18.54375 + 5 \times 0.028151 = 18.54375 + 0.140755 = 18.684505$$

Step 3, from T_2 to T_3 . The same four operations are repeated with the new temperature:

$$T_{\text{out}} - T_2 = -3.5 - 18.684505 = -22.184505$$

$$\frac{-22.184505}{240} = -0.092435, -0.092435 + 0.12 \times 1 = 0.027565$$

$$T_3 = 18.684505 + 5 \times 0.027565 = 18.684505 + 0.137825 = 18.82233$$

After three steps, 15 minutes, the model predicts $T_3 = 18.82233^\circ\text{C}$.

Step 4. Comparison with the exact solution. The final temperature with the heater on is

$$T_\infty = -3.5 + 0.12 \times 240 = -3.5 + 28.8 = 25.3$$

and the exact temperature after $t = 15$ minutes is

$$T(15) = 25.3 + (18.4 - 25.3)e^{-15/240} = 25.3 + (-6.9) \times 0.939413 = 25.3 + (-6.48195) = 18.81805$$

where $e^{-15/240} = e^{-0.0625} = 0.939413$. The Euler result is higher by $18.82233 - 18.81805 = 0.00428^\circ\text{C}$, a small error that shrinks with a shorter step Δt .

Step 5. Time to reach 21°C . With $T_{\text{target}} = 21$:

$$t = 240 \ln \left(\frac{18.4 - 25.3}{21 - 25.3} \right) = 240 \ln \left(\frac{-6.9}{-4.3} \right) = 240 \ln(1.604651) = 240 \times 0.472906 = 113.5 \text{ minutes}$$

The room needs almost two hours to warm up, which is why the predictive agent of the lab preheats before cold nights.

Worked example: How the dead band sets the switching rate

The hysteresis agent holds $T_{\text{set}} = 21^\circ\text{C}$ in the same room, with a constant outdoor temperature of -3.5°C and a 3 kW heater. The final temperatures are $T_{\infty, \text{on}} = -3.5 + 0.12 \times 240 = 25.3^\circ\text{C}$ with the heater on and $T_{\infty, \text{off}} = -3.5^\circ\text{C}$ with the heater off. Logarithms are rounded to six decimal places and times to four.

Step 1. A dead band of 1°C .

$$T_L = 21 - \frac{1}{2} = 20.5, T_H = 21 + \frac{1}{2} = 21.5$$

$$t_{\text{on}} = 240 \ln \left(\frac{20.5 - 25.3}{21.5 - 25.3} \right) = 240 \ln \left(\frac{-4.8}{-3.8} \right) = 240 \ln(1.263158) = 240 \times 0.233615 = 56.0676 \text{ min}$$

$$t_{\text{off}} = 240 \ln \left(\frac{21.5 - (-3.5)}{20.5 - (-3.5)} \right) = 240 \ln \left(\frac{25}{24} \right) = 240 \ln(1.041667) = 240 \times 0.040822 = 9.7973 \text{ min}$$

$$t_c = 56.0676 + 9.7973 = 65.8649 \text{ min}$$

$$\text{cycles per day} = \frac{1440}{65.8649} = 21.8629, \text{ switches per day} = 2 \times 21.8629 = 43.7258$$

$$D = \frac{56.0676}{65.8649} = 0.8513, E_{\text{day}} = 24 \times 0.8513 \times 3 = 61.29 \text{ kWh}$$

The heater switches about 44 times a day.

Step 2. A dead band of 0.2 °C. The same formulas give

$$T_L = 21 - \frac{0.2}{2} = 20.9, T_H = 21 + \frac{0.2}{2} = 21.1$$

$$t_{\text{on}} = 240 \ln \left(\frac{20.9 - 25.3}{21.1 - 25.3} \right) = 240 \ln \left(\frac{-4.4}{-4.2} \right) = 240 \ln(1.047619) = 240 \times 0.04652 = 11.1648 \text{ min}$$

$$t_{\text{off}} = 240 \ln \left(\frac{21.1 - (-3.5)}{20.9 - (-3.5)} \right) = 240 \ln \left(\frac{24.6}{24.4} \right) = 240 \ln(1.008197) = 240 \times 0.008164 = 1.9594 \text{ min}$$

$$t_c = 11.1648 + 1.9594 = 13.1242 \text{ min}$$

$$\text{cycles per day} = \frac{1440}{13.1242} = 109.721, \text{ switches per day} = 2 \times 109.721 = 219.442$$

$$D = \frac{11.1648}{13.1242} = 0.8507, E_{\text{day}} = 24 \times 0.8507 \times 3 = 61.25 \text{ kWh}$$

Step 3. Comparison. A dead band five times narrower makes the relay switch about five times more often, 219 instead of 44 times a day, while the duty cycle and the daily energy hardly change. The narrow band buys a smaller temperature swing with wear of the relay and the boiler. Sensor noise and delays, which the formulas ignore, make a band of zero far worse still, because the relay then chatters at every crossing of the set point [36]. Part D of the [interactive lab](#) computes these quantities for any band, room and heater.

A map of the course: Families of techniques

The course is organised around the kinds of engineering questions that artificial intelligence techniques answer. Search and planning answer the question of which sequence of actions reaches a goal at least cost [11]. Knowledge-based systems encode expert rules and explain their conclusions [12]. Fuzzy logic represents vague linguistic knowledge, such as a slightly high temperature, and turns it into smooth control actions [13, 37]. Evolutionary and swarm algorithms search large design spaces when gradients are unavailable [14, 19]. Machine learning builds models from data for prediction, classification and anomaly detection [2]. Deep learning learns representations from raw signals and images [24]. Generative models and large language models produce text, code and designs [26], and reinforcement learning learns control policies by interaction [32].

Weeks	Technique family	Typical engineering question
1 to 2	Agents, search and planning	Which route, sequence or plan reaches the goal at least cost?
3	Knowledge-based systems	Which diagnosis or class follows from expert rules and standards?
4	Fuzzy logic and fuzzy control	How can operator knowledge in words become a controller?
5 to 6	Evolutionary and swarm optimisation	Which design or tuning is best when the objective is a black box?
7 to 9	Machine learning	What will this material, machine or process do, and is it abnormal?
10 to 12	Deep learning for tables, images and sequences	What does this image, signal or time series show?
13	Language models and generative AI	How can text, code and knowledge bases support engineering work?
14	Learning to act, physics-informed learning, responsible AI	How can a system learn control, respect physics and remain accountable?

Check your understanding. A team must choose the order in which a drilling machine visits 200 holes on a circuit board. Which technique family fits the question best?

- A. Fuzzy logic
- B. Search and optimisation
- C. Image classification with deep learning
- D. Knowledge-based diagnosis

Answer: B. The question asks for a best sequence, which is a search or combinatorial optimisation problem, treated in Weeks 2, 5 and 6.

Artificial intelligence across the engineering disciplines

Recent results show how far the techniques of this course reach. AlphaFold predicts protein structures with accuracy close to experiment, a result that changed structural biology and drug design [29]. GraphCast, a graph neural network trained on reanalysis data, produces ten-day global weather forecasts that compete with the leading numerical models [30]. Deep reinforcement learning has controlled the magnetic coils that shape plasma in a tokamak [38], and graph networks have proposed hundreds of thousands of new stable inorganic crystals [39]. These systems combine the same building blocks that students meet here: data, models, optimisation, evaluation and domain knowledge.

Most engineering applications are less spectacular and more common. Mechanical engineers predict failures from vibration and temperature, civil engineers estimate concrete strength from mix proportions, food engineers grade grains from images, environmental engineers forecast emissions, and mining engineers anticipate seismic hazards in underground workings. The course uses public datasets from these fields, several of them produced in Türkiye, so that every department meets its own problems. Türkiye's National Artificial Intelligence Strategy names the training of a workforce able to apply artificial intelligence in all sectors as a strategic priority [40], which is the aim of an engineering course of this kind.

How to study this course

Every week follows the same cycle of reading, exploring, building, checking and reflecting. The lecture page contains animations and knowledge checks, the lab turns one idea into a simulation, and the Colab notebook rebuilds the idea in Python with real or realistic data. The overview page of each week connects these materials: It states the learning outcomes and the study path and lists the discipline challenges and the tasks. Earlier work on intelligent support for self-learning in computer engineering courses showed the value of matching materials and pacing to the learner when face-to-face time is limited [41]. The course therefore teaches Python in fourteen steps, one per week, and marks every further piece of Python in the notebooks, so that students without programming experience can learn the language while solving engineering problems, and students with experience can move quickly to the challenges. Each step opens the Colab notebook of its week and is printed with its outputs at the end of the lecture notes.

Engineering judgement: When not to use artificial intelligence

A well-validated physical model, a standard or a simple rule often beats a learned model, because it extrapolates, it is explainable and it needs no data. Artificial intelligence earns its place when the physics is unknown or too expensive to simulate, when expert knowledge is vague, when the search space is too large for enumeration, or when perception from images, signals or text is required. Even then, the system must be tested against the performance measure, its failure modes must be understood, and a person must remain accountable for its use [42, 43]. Training large models also has an energy cost that belongs in the engineering trade-off [44]. Week 14 returns to these questions with safety, standards and regulation.

Check your understanding. In which situation is a learned model least justified?

- A. A validated closed-form model predicts the quantity accurately across the operating range
- B. Defects must be recognised in camera images
- C. The design space has millions of combinations and no gradient
- D. Operators describe a control strategy only in vague words

Answer: A. When a validated physical model already works, a data-driven model adds cost and risk without a clear benefit.

Python, programming environments and Google Colab

Python is a general-purpose, high-level programming language created by Guido van Rossum in the early 1990s and developed today as open source software under the Python Software Foundation [45]. Its syntax is compact and readable, blocks are marked by indentation, and programs run without a separate compilation step. The main reason for its role in artificial intelligence is its ecosystem: The libraries used in this course, NumPy for arrays [46], pandas for tables [47], Matplotlib for figures [48], scikit-learn for machine learning [49] and PyTorch for deep learning [50], are all Python libraries. The official tutorial is the reference for the language itself [5].

Python code can be written and run in several environments. The interactive interpreter executes one line at a time and suits quick calculations. Scripts are text files with the extension .py that run from a terminal, which suits programs that run repeatedly or on servers. Integrated development environments such as Visual Studio Code, PyCharm and Spyder combine an editor, a debugger and project tools. Notebooks, such as Jupyter Notebook and JupyterLab, mix explanatory text, code, results and figures in one document, which makes them well suited to learning, exploration and reproducible reports [51]. Distributions such as Anaconda install Python together with the scientific libraries on a personal computer.

Google Colaboratory, Colab for short, is the environment of this course. It is a hosted Jupyter notebook service that runs in a web browser, needs no installation and gives free access to computing resources, including graphics processors, within usage limits [52]. The Open in Colab badge of every week opens the notebook directly from the course repository on GitHub. A notebook runs on a virtual machine in the cloud, the runtime, which comes with Python and the common libraries installed; further packages can

be added with a cell such as `!pip install ucimlrepo`. The runtime is temporary: It stops after a period of inactivity or when it reaches a maximum lifetime [52], and files written to it disappear with it. Work is therefore kept with File > Save a copy in Drive, and a graphics processor is selected with Runtime > Change runtime type when a notebook trains larger networks.

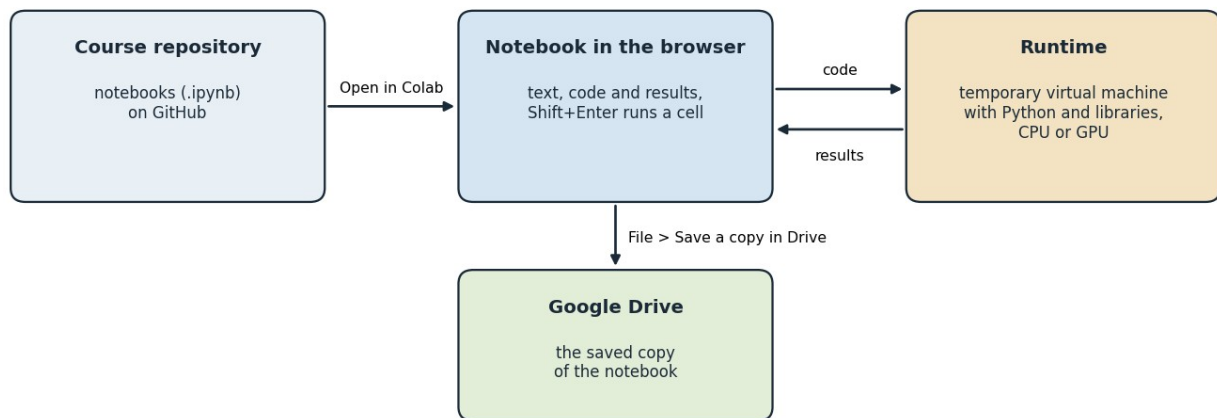


Figure 1.5. Working with the course notebooks in Google Colab [52].

A first session takes about fifteen minutes. Open the notebook of this week with its badge, sign in with a Google account and save a copy in Drive, so that changes are kept. Run the cells from the top with Shift+Enter and read each output before moving on. When a notebook stops behaving as expected, restarting the runtime from the Runtime menu and running all cells again from the top returns it to a clean state. Python step 1, printed at the end of these notes and taught at the start of this week's notebook, opens the Python strand of the course, one step per week.

Check your understanding. A student saves a results file in a Colab notebook, closes the browser and returns the next day. The file is gone. Why?

- A. Colab deletes files that contain errors
- B. The runtime is a temporary virtual machine, and its files are lost when the session ends
- C. Colab can only save files in PDF format
- D. The cells were not run with Shift+Enter

Answer: B. Files on the runtime disappear with the virtual machine. Lasting work belongs in Google Drive or in a repository.

Python step 1: Values, variables and decisions

Cells, values and variables

A notebook is a sequence of cells. Text cells explain, and code cells hold Python instructions that run when the cell is executed with Shift+Enter; the output appears directly below the cell [51]. The first building block of every program is the variable, a name that refers to a value. The equals sign assigns: `room_C = 18.4` stores the value 18.4 under the name `room_C`. It does not state an equation, so `count = count + 1` is a valid instruction that increases a counter. Names may contain letters, digits and underscores, cannot start with a digit and are case sensitive. Writing the unit into the name, as in `room_C` or `dt_min`, prevents many engineering mistakes, because Python itself does not track units.

```

setpoint_C = 21.0      # desired room temperature in degrees Celsius
room_C = 18.4          # measured room temperature
outside_C = -3.5       # outside air temperature
heater_on = False      # state of the heater
print(type(setpoint_C), type(heater_on))
print("room:", room_C, "outside:", outside_C)
  
```

Output

```
<class 'float'> <class 'bool'>  
room: 18.4 outside: -3.5
```

Every value has a type. Numbers with a decimal point are floats, whole numbers are integers, text in quotation marks is a string, and `True` and `False` are booleans. The function `type` reports the type, and `print` shows its arguments separated by spaces. Text after a hash sign is a comment for human readers and is ignored by Python.

Arithmetic and one step of the room model

Python uses the usual operators `+`, `-`, `*` and `/`, with `**` for powers, `//` for division rounded down and `%` for the remainder. Multiplication and division bind more strongly than addition and subtraction, and parentheses override the order. The room of this week loses heat to the outside in proportion to the temperature difference and gains heat while the heater runs. With a time constant τ , a heating rate r and a time step dt , one step of the model reads $T_{\text{next}} = T + (T_{\text{out}} - T) dt / \tau + r dt u$, where u is 1 when the heater is on and 0 otherwise.

```
tau_min = 240          # time constant of the room in minutes  
rate_C_per_min = 0.12 # warming rate while the heater runs  
dt_min = 5            # length of one time step in minutes  
u = 1                # heater command: 1 for on, 0 for off  
room_next = room_C + (outside_C - room_C) * dt_min / tau_min +  
rate_C_per_min * dt_min * u  
print(room_next)  
minutes = 290  
print(minutes // 60, "hours and", minutes % 60, "minutes")
```

Output

```
18.54375  
4 hours and 50 minutes
```

The printed temperature may carry a small rounding residue in its last digits. Computers store decimal fractions in binary, so values such as 0.1 are close approximations. The differences lie far below any measurement precision, but they are the reason why floats should not be compared with `==`.

Text and f-strings

Results need clear labels and units. An f-string, a string with the letter `f` before the opening quotation mark, inserts the value of any expression written in braces. A format specification after a colon controls the display: `:.2f` shows two decimals, `:+.2f` also shows the sign, and `:.0%` shows a fraction as a percentage.

```
print(f"Next temperature: {room_next:.2f} °C")  
print(f"Change in one step: {room_next - room_C:+.2f} K")  
print(f"Share of the hour with heating: {0.37:.0%}")
```

Output

```
Next temperature: 18.54 °C  
Change in one step: +0.14 K  
Share of the hour with heating: 37%
```

Comparisons and decisions

Comparisons such as `<`, `<=`, `>`, `>=`, `==` (equal) and `!=` (not equal) produce booleans, and `and`, `or` and `not` combine them. Python also accepts chained comparisons such as `low_C <= room_C <= high_C`, which read like mathematical notation. An `if` statement runs its indented block only when its

condition is true; `elif` adds further conditions, and `else` covers all remaining cases. The colon at the end of each header line and the indentation of four spaces are part of the syntax: Indentation decides which lines belong to a block. The thermostat rule of this week, with a band from 20.5 to 21.5 °C, becomes the following code.

```
low_C, high_C = 20.5, 21.5
comfortable = low_C <= room_C <= high_C
print("comfortable:", comfortable)

if room_C < low_C:
    heater_on = True
elif room_C > high_C:
    heater_on = False
else:
    pass # inside the band: keep the previous state
print("heater on:", heater_on)
```

Output

```
comfortable: False
heater on: True
```

The line `low_C, high_C = 20.5, 21.5` assigns two values at once. The statement `pass` does nothing. It marks the place where the previous state is deliberately kept, which is the hysteresis of the agent.

Repeating a step

A `for` loop repeats its indented block once for every value of a sequence. `range(12)` produces the integers 0 to 11, so the loop below simulates one hour in twelve steps of five minutes. Each pass first applies the rule and then updates the temperature, and the conditional expression `1 if heater_on else 0` turns the boolean into the heater command `u`. Week 2 treats loops in more depth.

```
room = 18.4
heater_on = False
for step in range(12): # 12 steps of 5 minutes: one hour
    if room < low_C:
        heater_on = True
    elif room > high_C:
        heater_on = False
    u = 1 if heater_on else 0
    room = room + (outside_C - room) * dt_min / tau_min +
    rate_C_per_min * dt_min * u
print(f"after one hour: {room:.2f} °C, heater on: {heater_on}")
```

Output

```
after one hour: 19.94 °C, heater on: True
```

When Python cannot run a line, it stops and prints an error message whose last line names the problem. A `NameError` usually means that a variable is used before the cell that defines it has been run, and an `IndentationError` points to a block whose lines are not aligned. Reading the last line first saves a lot of time.

Check your understanding. What does `print(17 // 5, 17 % 5)` display?

- A. 3.4 0
- B. 3 2

C. 2 3
D. 3.4 2

Answer: B. Floor division gives the whole number of times 5 fits into 17, which is 3, and the remainder is 2.

Discipline challenges

Every department of the Faculty of Engineering and Natural Sciences meets agents in its own form. Choose the row of your department, or the closest one, and write the PEAS description and the environment properties of the system named there. Students from other programmes worldwide can use the closest discipline.

Department	Challenge
Computer Engineering	A network intrusion detection agent that watches traffic and blocks suspicious connections.
Biology	An automated microscope that finds and counts dividing cells in time-lapse images.
Environmental Engineering	An aeration controller in a wastewater treatment plant that keeps dissolved oxygen in range.
Electrical and Electronics Engineering	A battery management system that balances cells and limits charging current.
Industrial Engineering	A scheduling agent that reassigns jobs when a machine breaks down.
Physics	An autonomous telescope scheduler that selects targets as clouds move across the sky.
Food Engineering	A fruit-drying controller that adjusts air temperature from product moisture.
Civil Engineering	A structural health monitoring system on a bridge that raises alarms from strain gauges.
Statistics	An automated quality-control agent that flags anomalous values in incoming survey data.
Geophysical Engineering	An earthquake early-warning agent that issues alerts from the first seconds of shaking.
Geological Engineering	A core-logging camera that labels rock types along drill cores.
Chemistry	A self-driving laboratory robot that chooses the next experiment from previous results.
Chemical Engineering	A distillation column controller that holds product purity during feed changes.
Mining Engineering	An autonomous haul truck operating on open-pit roads with loaders and other trucks.
Mechanical Engineering	A condition monitoring agent for a pump that schedules maintenance from vibration.
Mathematics	A solver-selection agent that picks a numerical method for each differential equation it receives.
Automotive Engineering	An adaptive cruise control that keeps a safe gap to the vehicle ahead.
Textile Engineering	A fabric inspection camera that marks defects on a running loom.
Earth Sciences Engineering	A landslide monitoring network that raises warnings from displacement and rainfall sensors.

Weekly task

Write a short technical note of about 400 words that specifies an intelligent system from your own department with the PEAS scheme, classifies its environment with the six properties of the lecture, and names the agent architecture that fits it. Add the comparison table of the three thermostat agents from the notebook and one plot, and explain in two sentences which agent you would install and why. Cite at least three works from this week's references in square brackets [1, 34, 36].

Research and report assignment (optional)

Artificial intelligence in your discipline. Select one engineering or science department from the discipline table of this week and review how artificial intelligence is used in it today. Classify at least eight published applications by the technique families of the course map, by the type of data they use and by their environment properties. Use one recent review from the course bibliography as a starting point, for example the reviews on structural engineering, manufacturing, Earth science or materials [53, 54, 55, 56].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- [2] Jordan, M. I., & Mitchell, T. M. (2015). Machine learning: Trends, perspectives, and prospects. *Science*, 349(6245), 255-260. <https://doi.org/10.1126/science.aaa8415>
- [3] Hersbach, H., Bell, B., Berrisford, P., et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999-2049. <https://doi.org/10.1002/qj.3803>
- [4] Open-Meteo (2026). *Open-Meteo free weather API: Historical Weather API and Elevation API (data licensed under CC BY 4.0)*. <https://open-meteo.com>
- [5] Python Software Foundation (2026). *The Python Tutorial (Python 3 documentation)*. <https://docs.python.org/3/tutorial/>
- [6] Turing, A. M. (1950). Computing machinery and intelligence. *Mind*, 59(236), 433-460. <https://doi.org/10.1093/mind/LIX.236.433>
- [7] McCarthy, J., Minsky, M. L., Rochester, N., & Shannon, C. E. (2006). A proposal for the Dartmouth summer research project on artificial intelligence, August 31, 1955. *AI Magazine*, 27(4), 12-14. <https://doi.org/10.1609/aimag.v27i4.1904>
- [8] McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4), 115-133. <https://doi.org/10.1007/BF02478259>
- [9] Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3), 210-229. <https://doi.org/10.1147/rd.33.0210>
- [10] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408. <https://doi.org/10.1037/h0042519>
- [11] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107. <https://doi.org/10.1109/TSSC.1968.300136>

- [12] Buchanan, B. G., & Shortliffe, E. H. (Eds.) (1984). *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley.
- [13] Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338-353. [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X)
- [14] Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems* (2nd ed.). MIT Press. First edition 1975, University of Michigan Press.
- [15] Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- [16] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. <https://doi.org/10.1038/323533a0>
- [17] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297. <https://doi.org/10.1007/BF00994018>
- [18] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- [19] Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of ICNN'95, International Conference on Neural Networks*, Vol. 4 (pp. 1942-1948). IEEE. <https://doi.org/10.1109/ICNN.1995.488968>
- [20] Dorigo, M., Maniezzo, V., & Coloni, A. (1996). Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1), 29-41. <https://doi.org/10.1109/3477.484436>
- [21] Campbell, M., Hoane, A. J., & Hsu, F.-h. (2002). Deep Blue. *Artificial Intelligence*, 134(1-2), 57-83. [https://doi.org/10.1016/S0004-3702\(01\)00129-1](https://doi.org/10.1016/S0004-3702(01)00129-1)
- [22] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [23] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6), 84-90. <https://doi.org/10.1145/3065386>
- [24] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444. <https://doi.org/10.1038/nature14539>
- [25] Silver, D., Huang, A., Maddison, C. J., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484-489. <https://doi.org/10.1038/nature16961>
- [26] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998-6008). <https://arxiv.org/abs/1706.03762>
- [27] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33* (pp. 1877-1901). <https://arxiv.org/abs/2005.14165>
- [28] Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35* (pp. 27730-27744). <https://arxiv.org/abs/2203.02155>
- [29] Jumper, J., Evans, R., Pritzel, A., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583-589. <https://doi.org/10.1038/s41586-021-03819-2>
- [30] Lam, R., Sanchez-Gonzalez, A., Willson, M., et al. (2023). Learning skillful medium-range global weather forecasting. *Science*, 382(6677), 1416-1421. <https://doi.org/10.1126/science.adi2336>
- [31] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

- [32] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [33] Engelbrecht, A. P. (2007). *Computational Intelligence: An Introduction* (2nd ed.). Wiley.
<https://doi.org/10.1002/9780470512517>
- [34] Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2), 115-152. <https://doi.org/10.1017/S0269888900008122>
- [35] Wiener, N. (1961). *Cybernetics: Or Control and Communication in the Animal and the Machine* (2nd ed.). MIT Press.
- [36] Åström, K. J., & Murray, R. M. (2021). *Feedback Systems: An Introduction for Scientists and Engineers* (2nd ed.). Princeton University Press.
- [37] Mamdani, E. H., & Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1), 1-13. [https://doi.org/10.1016/S0020-7373\(75\)80002-2](https://doi.org/10.1016/S0020-7373(75)80002-2)
- [38] Degraeve, J., Felici, F., Buchli, J., et al. (2022). Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, 602(7897), 414-419. <https://doi.org/10.1038/s41586-021-04301-9>
- [39] Merchant, A., Batzner, S., Schoenholz, S. S., et al. (2023). Scaling deep learning for materials discovery. *Nature*, 624(7990), 80-85. <https://doi.org/10.1038/s41586-023-06735-9>
- [40] Presidency of the Republic of Türkiye Digital Transformation Office, & Ministry of Industry and Technology (2021). *National Artificial Intelligence Strategy 2021-2025 (Ulusal Yapay Zekâ Stratejisi 2021-2025)*. Ankara. Action plan updated for 2024-2025. <https://www.cbddo.gov.tr/UYZS>
- [41] Kose, U., & Arslan, A. (2017). Optimization of self-learning in computer engineering courses: An intelligent software system supported by artificial neural network and vortex optimization algorithm. *Computer Applications in Engineering Education*, 25(1), 142-156. <https://doi.org/10.1002/cae.21787>
- [42] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv preprint arXiv:1606.06565. <https://arxiv.org/abs/1606.06565>
- [43] Kose, U. (2018). Are we safe enough in the future of artificial intelligence? A discussion on machine ethics and artificial intelligence safety. *BRAIN. Broad Research in Artificial Intelligence and Neuroscience*, 9(2), 184-197.
- [44] Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3645-3650). <https://doi.org/10.18653/v1/P19-1355>
- [45] Python Software Foundation (2026). *History and license (Python 3 documentation)*.
<https://docs.python.org/3/license.html>
- [46] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- [47] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56-61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- [48] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- [49] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [50] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024-8035).
<https://arxiv.org/abs/1912.01703>

- [51] Kluyver, T., Ragan-Kelley, B., Pérez, F., et al. (2016). Jupyter Notebooks: A publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas* (pp. 87-90). IOS Press. <https://doi.org/10.3233/978-1-61499-649-1-87>
- [52] Google (2026). *Google Colaboratory: Frequently asked questions*. <https://research.google.com/colaboratory/faq.html>
- [53] Salehi, H., & Burgueño, R. (2018). Emerging artificial intelligence methods in structural engineering. *Engineering Structures*, 171, 170-189. <https://doi.org/10.1016/j.engstruct.2018.05.084>
- [54] Wuest, T., Weimer, D., Irgens, C., & Thoben, K.-D. (2016). Machine learning in manufacturing: Advantages, challenges, and applications. *Production & Manufacturing Research*, 4(1), 23-45. <https://doi.org/10.1080/21693277.2016.1192517>
- [55] Bergen, K. J., Johnson, P. A., de Hoop, M. V., & Beroza, G. C. (2019). Machine learning for data-driven discovery in solid Earth geoscience. *Science*, 363(6433), eaau0323. <https://doi.org/10.1126/science.aau0323>
- [56] Butler, K. T., Davies, D. W., Cartwright, H., Isayev, O., & Walsh, A. (2018). Machine learning for molecular and materials science. *Nature*, 559(7715), 547-555. <https://doi.org/10.1038/s41586-018-0337-2>